КРИТИЧНІ ОБЛАСТІ ТА МОНІТОРИ

Недоліки та обмеження семафорів (1)

- Незважаючи на простоту семафорів, їх використання для програмування синхронізації паралельних процесів пов′язано з певними *недоліками та обмеженнями.*
- 1. Неструктурованість. P- та V-операції, які мають аргументом один і той ж семафор, можуть бути розподілені по різним процесам і програмним модулям, що вносить додаткову складність у процеси розробки цих модулів.
- 2. Можливості легко помилитись при забезпеченні взаємного виконання. Наприклад, випадкова зміна місць P- та V-операцій, що не зразу впадає в око:
  - V (mutex);
  - КС;
  - P (mutex)
- означатиме *одночасний вхід в КС* декількох процесів.

Недоліки та обмеження семафорів (2)

- 3. Можливість *тупиків:* наприклад, програмний код
  - P(mutex);
  - КС;
  - P(mutex)
- призведе до того, що ресурс не буде звільнений після виконання КС, а значить процеси, що чекали на його звільнення, ніколи не увійдуть в свої критичні секції (*тупик за ресурсами*).
- 4. Семафори *надто примітивні* конструкції для забезпечення складних дисциплін синхронізації, що вимагають перевірки не однієї, а декількох змінних (семафорів).

Засіб критичної області

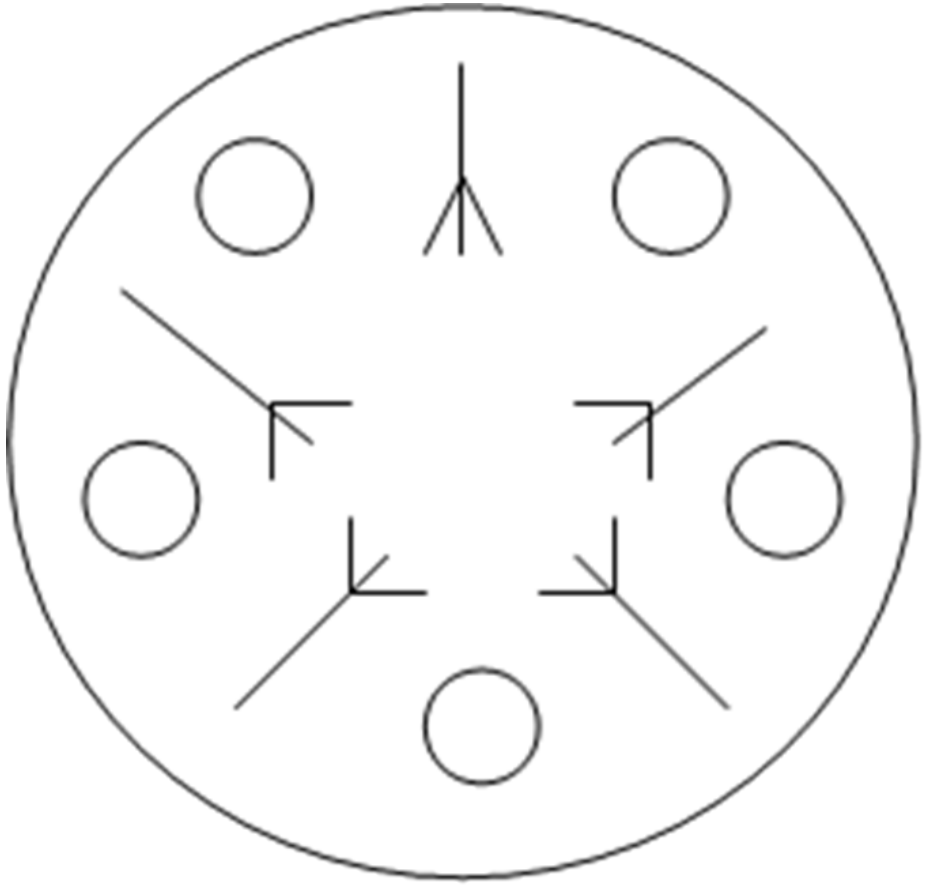
- Для подолання цих недоліків були запропоновані синхронізаційні примітиви *більш високого рівня*.
- Один з таких примітивів – *критична область*. У фрагменті програми нижче, зліва подано опис критичної секції за допомогою критичної секції ***r***, а справа – еквівалентний запис за допомогою семафора:
- var ***r***: shared T ; P(r);
- … S
- region ***r*** do S; V(r)
- де S — послідовність операторів, що є *критичною секцією* і виконує дії над спільним об’єктом типу T у взаємовиключному режимі.
- На відміну від семафорів, критична область пов’язує себе з відповідним ресурсом *явним чином* і реалізує це єдиною операцією region.

Умовна критична область

- Удосконалення критичної секції – *умовна критична область* виду
  - region r when B do S
- означає, що процес мусить перевіряти умову B, і лише, якщо B=true, може виконуватись критична секція S інакше процес відкладається.
- Засоби критичних областей можна продемонструвати на прикладі ще однієї відомої задачі паралельного програмування — "філософів, що обідають" (dining philosophers)

Задача про "філософів, що обідають"

- Це приклад проблеми програмування спільних ресурсів, зміст якої полягає у тому, щоб запропонувати протокол взаємодії філософів такий що:
  - забезпечується взаємне виключення при користуванні парою виделок;
  - не повинно бути тупика при використанні виделок (спільних ресурсів).



Варіант використання засобів семафорів

- Якщо для синхронізації філософів скористатися засобами семафорів, то легко побачити, що відповідна програма:
  - fork [0..4] of <u>binary semaphores</u>;
  - fork [0..4] := [1,1,11,1];
  - <u>repeat</u>: think;
  - P(fork[i]);
  - P(fork ([i ⊕ 1]);
  - eat;
  - V (fork [i]);
  - V (fork [i ⊕ 1])
  - <u>end</u>;
- *попадає в тупик*, якщо кожний з філософів одночасно візьме одну (ліву) виделку.
- Уникнути цієї ситуації за допомогою семафорів неможливо, бо вони нездатні перевіряти складну умову *одночасної наявності двох виделок* на одному місці.

Ідея використання умовних критичних областей (1)

- Ідея розв’язування цієї задачі для двох процесів з використанням умовних критичних областей має вигляд:
  - region r do
  - begin S1;
  - await (B);
  - S2;
  - End
- де S1 і S2 — критичні області (можливо пусті). Після виконання S1, якщо B=false, процес відкладається, звільняючи ресурс r від взаємного виключення.

Ідея використання умовних критичних областей (2)

```
var r shared record fork: array [0..4] of integer;  i:integer;  fork [0,1,2,3,4]:= [2,2,2,2,2,];
(*philosopher i *)
repeat
        think;
                region r when (fork[i]=2) do
                  begin
                        fork [i ⊕ 1] -1;
                        fork (i θ 1] -1;
        end
                eat;
                region r do begin
                        fork [i ⊕ 1] := fork [i ⊕ 1] +1;
                        fork [i θ 1] := fork [i θ 1] -1;
                end
        forever
```

Монітори Хоора (A. Hoare)

- Ідея монітора полягає у тому, щоб "зібрати" в ньому всі критичні секції процесів і акумулює їх у собі у вигляді процедур або функцій, що можуть виконуватись у *взаємовиключному режимі*.
- У цьому сенсі, монітори є *найбільшим структурованим* засобом синхронізації та взаємовиключення процесів над спільною пам′ятью.
- Синтаксична структура монітора як засобу програмування виглядає таким чином:
  - monitor < ім′я -монітора >;
  - (* опис локальних даних *)
  - procedure <ім′я>(< список параметрів>);
  - …
  - procedure …
  - begin (* ініціалізація локальних даних*) end.

Типи змінних і операції моніторів

- Процедури можуть оперувати тільки з *локальними змінними* монітора.
- Синхронізація досягається за допомогою двох спеціальних операцій WAIT(c) і SIGNAL(c), що виконуються над змінною типу CONDITION, що на загал інтерпретується як умова пустоти чи непустоти черги.
- Виклик процедури монітора проводиться через вказівку імені монітора, тобто: < ім’я - монітора >.< ім’я - процедури >(<список - параметрів >.

Моделювання бінарного семафора через монітор

- Бінарний семафор визначає взаємовиключний доступ до деякого ресурсу.
  - monitor S;
  - var busy: bool;
  - not: condition;
  - procedure P;
  - begin
  - if busy then wait(not); busy:= true;
  - end
  - procedure V;
  - begin
  - busy := false; signal(not);
  - end
  - begin busy := false; end
- Щоб скористатись цією програмою для виконання P-операції треба виконати виклик процедури монітора S.P(x), а для виконання V-операції – процедури S.V(x).

Монітор для обмеженого буфера на N місць

- monnitor: b_buffer; var prt, count: integer; buf: array {0.. n-1} of item;
- nonfull, nonempty: condition;
- procedure put (x:item)
- wait (nonfull);
- buf[count ⊕ ptr] : = x;
- count := count + 1;
- signal(nonemmpty); end;
- procedure get(x: item);
- wait (nonempty);
- x:= buf [ptr];
- count := count -1;
- signal (nonfull); end;
- begin ptr, count := 0; end.